

RaftScope: A Causally-Instrumented Raft Implementation with Space-Time Visualization

CSCI 546, Distributed Systems, Spring 2026

Karthik Venugopal Akshay Shivashankar Bharadwaj Adithya Sudhindra
University of Southern California

1 Introduction

This report describes **RaftScope**, our final project for CSCI 546. The system has two components that together enable principled debugging of distributed consensus: (1) a complete C++ implementation of the Raft consensus algorithm built on the course skeleton using gRPC, and (2) a self-contained, browser-based space-time diagram visualizer that reconstructs the causal history of a run from per-node JSONL event logs.

The core thesis is that Raft correctness bugs (split votes, log divergence, liveness stalls) are most naturally diagnosed by inspecting the *causal* ordering of messages across nodes, not by correlating wall-clock timestamps from separate log files. RaftScope embeds Lamport timestamps in every RPC so the visualizer can draw causal arrows between events even when system clocks skew.

All six required integration tests pass (InitialElectionA, ReElectionA, ManyElectionA, LeaderStepsDownOnHigherTermA, RapidLeaderChurnA, ExampleTestA).

2 Background

2.1 Raft Consensus

Raft [1] is a leader-based consensus algorithm designed for understandability. A cluster of n nodes tolerates up to $\lfloor (n - 1)/2 \rfloor$ simultaneous failures. All writes are serialized through a single elected leader: clients submit entries to the leader, which replicates them to followers via `AppendEntries` RPCs. An entry is committed once a majority of nodes have durably stored it and the leader has advanced its `commit_index`. Leader election uses

majority vote with a log up-to-date check that guarantees no committed entry can be lost across a leadership change.

2.2 Lamport Logical Clocks

Lamport [2] showed that a simple scalar clock suffices to assign a total order consistent with causality in a message-passing system. The rule is: (1) increment the clock on every local event or outgoing message; (2) on receiving a message, set the clock to $\max(\text{local}, \text{received}) + 1$. This ensures every send has a strictly smaller timestamp than its matching receive. Unlike wall-clock timestamps, Lamport timestamps are immune to clock skew and NTP drift, making them the right primitive for reconstructing causal order across nodes with no shared time source. RaftScope embeds a Lamport timestamp in every non-heartbeat RPC so the visualizer can draw causal arrows that are provably ordered correctly regardless of host clock accuracy.

3 Design

3.1 Raft State Machine

We implement the full Raft state machine from the extended Raft paper [1]. Each node is always in one of three roles: FOLLOWER, CANDIDATE, or LEADER. Role transitions are:

- **Follower** $\rightarrow$ **Candidate**: election timer fires without receiving a heartbeat or granting a vote within the randomized `election_timeout_ms` window (250–500 ms, seeded per-node to break symmetry).
- **Candidate** $\rightarrow$ **Leader**: majority of peers grant the vote in the same term.

- **Leader/Candidate → Follower:** any incoming RPC carries a term $>$ current term.

All Raft state (`current_term`, `voted_for`, `log`, `commit_index`, `last_applied`, `next_index`, `match_index`) is protected by a single mutex `mtx`. RPC handlers hold this mutex for their entire execution. Background threads release it before making outgoing gRPC calls, preventing a stalled peer from blocking the local server.

3.2 RPC Protocol

We define two RPCs in `proto/raft.proto`: `RequestVote` (candidate → peers) and `AppendEntries` (leader → followers, also used as heartbeats when `entries` is empty). Both request and reply messages carry a `lamport_ts` field that threads the Lamport clock through the network.

`AppendEntriesReply` also carries `conflict_term` and `conflict_index` to support the fast-backup optimization (Section 4.3).

3.3 RaftScope Instrumentation Layer

The `RaftEventLogger` class (205 lines in `inc/rafty/raft_scope.hpp`) attaches to each Raft node and writes newline-delimited JSON (JSONL) events to `logs/raft_scope_node_{id}.jsonl`. Every event record contains:

- `lamport_ts`: the Lamport clock value at event time
- `wall_time_us`: wall-clock microseconds (for layout only)
- `node_id`, `from_node`, `to_node`: causal endpoints
- `event_type`: one of 15 named types (`vote-request`, `vote-granted`, `log-append`, `commit`, `leader-change`, `node-crash`, `partition-start/end`, ...)
- `term`, `log_index`, `commit_index`, `role`: snapshot of consensus state

The logger owns a `LamportClock`: `tick()` on send (increments by 1), `update(received)` on receive (sets to $\max(\text{local}, \text{received}) + 1$ via a compare-exchange loop for thread safety). Heartbeat `AppendEntries` advance the clock but do not write an event record, keeping logs focused on meaningful state changes.

3.4 Space-Time Visualizer

The visualizer (`visualizer/index.html`) is a single self-contained HTML file using D3.js v7. It accepts multi-file JSONL drag-and-drop (or file picker) and renders:

- A vertical timeline per node, with events placed by their Lamport timestamp on the *y*-axis.
- Causal arrows from `from_node` at the send event to `to_node` at the matched receive event, colored by event type.
- A click-to-expand event detail panel showing all fields.
- Checkboxes to filter event types (e.g., hide heartbeats, show only commit/leader-change events).
- Live auto-refresh mode: polls the log files every 2 seconds and animates newly arrived events with a CSS “pop-in” keyframe, enabling real-time observation of a running cluster.

4 Implementation

4.1 Background Threads

Three detached threads are started by `Raft::run()`:

election_thread_ Sleeps in 10 ms increments. If elapsed time since `last_heartbeat_exceeds_election_timeout_ms` and the node is not a leader, it atomically increments the term, sets role to CANDIDATE, and fires a parallel wave of `RequestVote` RPCs (one thread per peer, each with a 200 ms deadline).

heartbeat_thread_ Waits on `heartbeat_cv_` with a 50 ms timeout. When the node is leader it spawns one thread per peer to call `send_append_entries_to`. The `propose()` function notifies this condvar to trigger immediate replication without waiting the full interval.

apply_thread_ Waits on `apply_cv_` until `last_applied < commit_index`. Drains the committed suffix into a local vector under the lock, then calls `apply()` outside the lock to avoid deadlock with the framework’s message-queue.

4.2 Log Replication and Consistency Check

`AppendEntries` performs the standard two-phase log consistency check: it rejects the request if `prev_log_index` is beyond the local log length, or if `log[prev_log_index].term` does not match `prev_log_term`. When appending, it truncates any conflicting suffix before inserting new entries, preserving the log matching property.

Commit advancement uses the standard majority rule: the leader scans backwards from the last log index looking for the highest N such that `log[N].term == current_term` and a majority of `match_index[i] >= N`.

4.3 Fast-Backup Optimization

Naive log replication decrements `next_index` by one per rejected `AppendEntries`, requiring $O(\text{stale_entries})$ round trips after a partition heals. We implement the fast-backup protocol: on rejection the follower returns the term of the conflicting entry (`conflict_term`) and the first index of that term (`conflict_index`). The leader searches its log for the last entry with `conflict_term`; if found it sets `next_index` to just past it, otherwise it uses `conflict_index` directly. This lets a rejoining node catch up in a single round trip per stale term rather than per stale entry.

4.4 Implementation Challenges

The most subtle aspect of the implementation was maintaining correct locking discipline across gRPC calls. Holding `mtx` while making an outgoing RPC deadlocks if the remote peer concurrently calls back into the same node (e.g., a simultaneous `AppendEntries` and `RequestVote` crossing on the wire). We resolved this by copying all needed state under the lock, releasing it, making the network call, then re-acquiring the lock to process the reply. This pattern appears in every outgoing RPC in `src/raft.cpp`.

A second challenge was the `LamportClock::update` implementation. A mutex-based approach would serialize all receive-side clock updates, bottlenecking concurrent RPC handlers. We instead use a compare-exchange loop: read the current value, compute $\max(\text{current}, \text{received}) + 1$, and install atomically only if the value has not

changed. This is correct under any level of concurrency and avoids adding a second lock to the critical path.

4.5 Safety Properties Maintained

- **Election safety:** a node only votes once per term (`voted_for_` persists until a term change) and only for candidates whose log is at least as up-to-date as its own (comparison by `last_log_term` then `last_log_index`).
- **Log matching:** enforced by the consistency check and truncation-before-append logic.
- **Leader completeness:** the log up-to-date check in `RequestVote` prevents a candidate missing committed entries from winning an election.
- **Granting a vote resets the election timer:** prevents a freshly voted-for candidate from being immediately challenged.

5 Evaluation

5.1 Correctness: Integration Tests

All six required integration tests pass consistently:

Test	Nodes	Result
InitialElectionA	3	pass
ReElectionA	3	pass
ManyElectionA	7	pass
LeaderStepsDownOnHigherTermA	3	pass
RapidLeaderChurnA	5	pass
ExampleTestA	3	pass

The election timeout range of 250–500 ms with a 50 ms heartbeat gives a timeout-to-heartbeat ratio of 5–10 $\times$, well above the recommended lower bound to avoid false timeouts on a loaded test machine. Election timeouts are seeded per-node using `std::random_device` XORed with the node ID, preventing all nodes from timing out simultaneously and causing repeated split votes.

The most demanding test is `ManyElectionA`, which runs 7 nodes and repeatedly disconnects and reconnects random subsets, exercising split votes, rapid term increments, and nodes rejoining with stale logs simultaneously. Our implementation passes this test consistently across 20 repeated runs with no observed liveness violations.

5.2 Observability: Space-Time Diagram

To exercise the visualizer we ran a 5-node cluster under `scripts/run_fault_injection.sh`, which injects random kills and network partitions over 60 seconds. The resulting JSONL logs (one per node) were dropped into the visualizer. We confirmed:

1. Causal arrows are consistent: every `vote_request` arrow terminates at the corresponding `vote_recv` on the peer, at a Lamport time strictly greater than the send.
2. Leader-change events cluster at term boundaries, making split-vote rounds visually obvious.
3. Commit events on the leader appear after the corresponding `log_append` arrows reach a majority of followers, verifiable by inspection.
4. The live-refresh mode correctly streams new events into the diagram without re-rendering existing structure, confirmed by a 30-second real-time observation session.

Debugging a liveness bug. During development, `LeaderStepsDownOnHigherTermA` failed intermittently. The visualizer made the root cause immediately apparent: after a leader stepped down on seeing a higher term, `voted_for_` was not reset to `-1`. The ex-leader therefore refused to vote for any candidate in the new term, believing it had already voted, and the cluster could not form a new majority. The bug appeared in the diagram as a term boundary where `vote_request` arrows arrived at the ex-leader but produced no corresponding `vote_granted` or `vote_denied` arrows outward. Correlating this pattern across five raw log files without the visualizer would have required significant manual effort.

5.3 Performance

We measured round-trip latency for log replication on a single machine (loopback gRPC) at approximately 1–3 ms per entry. The 50 ms heartbeat interval means commits typically land within one heartbeat interval of a `propose()` call. The fast-backup optimization was validated by the `BackupExtraB` test, which requires a rejoining follower with hundreds of stale entries to catch up within a strict RPC-count budget; naive decrement-by-one would exceed

that budget by an order of magnitude.

The Lamport timestamp overhead is negligible: one additional 64-bit integer field per RPC adds at most 8 bytes to each protobuf payload, well within gRPC’s default 4 MB message limit. We verified that enabling the instrumentation did not affect election timing by running the full test suite with and without the `scope_logger` active; pass rates and wall-clock test durations were statistically indistinguishable across 10 runs each.

6 Conclusion

RaftScope demonstrates that embedding causal clocks directly into the consensus protocol (at the RPC field level) adds minimal overhead (one 64-bit integer per message) while enabling rich post-hoc and real-time debugging through a browser-based visualizer. The implementation passes all required correctness tests and the instrumentation layer does not interfere with timing-sensitive election behavior.

References

- [1] Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Extended Version)*. USENIX ATC 2014.
- [2] Leslie Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 21(7), 1978.

AI Usage Report

Overview

We used **Claude Code** (Anthropic’s CLI agent, powered by the `claude-sonnet-4-6` model) to build the RaftScope visualizer (`visualizer/index.html`). Claude Code differs from a chat assistant in that it can read, write, and execute code directly in the project directory and observe the output, behaving more like a pair-programmer with shell access than a question-answering tool.

What the AI Built

We described the desired space-time layout (nodes as columns, Lamport time as vertical axis, causal arrows between matching send/receive events) and Claude Code produced the full implementation: the `D3.js v7` rendering loop, the multi-file JSONL drag-and-drop parser, the event-type color map, the click-to-expand sidebar panel, and the checkbox filter controls. The live auto-refresh mode (polling log files every 2 seconds and animating newly arrived events with a CSS `pop-in` keyframe) was also AI-generated in a single prompt round.

What Went Well

Speed of scaffolding. The D3 boilerplate, SVG coordinate math, and file-drop handler would have taken several hours to write by hand. Claude Code produced a working first draft in under ten minutes, letting us focus on what the diagram should *show* rather than how to wire up the DOM.

Incremental feature additions. Adding the live-refresh mode, the event-detail sidebar, and the arrow-label overlays each required a single focused prompt. The AI preserved the existing structure and spliced in only the new logic, which made reviewing diffs straightforward.

What Did Not Work Well

Visual layout tuning. The AI cannot see the rendered result. Iterating on spacing, font sizes, arrow curvature, and legend placement required us to open the browser, observe what looked wrong, describe it in text, and repeat. Several rounds were needed for the causal arrow routing in particular: the first draft drew arrows that crossed node timelines in confusing

ways.

Edge-case JSONL handling. The initial parser crashed silently on malformed or truncated lines (which occur when a node is killed mid-write). We had to explicitly describe the failure mode before the AI added the `try/catch` guard around each `JSON.parse` call.

Overall Assessment

Claude Code was the right tool for the visualizer. The component is primarily JavaScript and CSS with a clear, self-contained spec, so the AI could generate correct code quickly and iterative refinement was cheap. The main limitation (not being able to see the rendered output) slowed layout work, but for logic and structure the collaboration was efficient.

Appendix: Representative Prompts

Prompt 1 (initial visualizer):

“Build a single self-contained HTML file using D3.js v7. It accepts a drag-and-drop of multiple JSONL files (one per Raft node). Each line is a JSON object with fields: `lamport_ts`, `node_id`, `event_type`, `from_node`, `to_node`, `term`, `log_index`, `commit_index`, `role`. Render a space-time diagram: one vertical timeline per node (x-axis), events placed by `lamport_ts` on the y-axis as colored circles. Draw causal arrows from the send event on `from_node` to the matching receive event on `to_node`. Include a sidebar that shows all fields for the clicked event.”

Prompt 2 (live auto-refresh):

“Add a live auto-refresh mode to the visualizer. Every 2 seconds, re-read the JSONL files (same paths as last load). Only render events with `lamport_ts` greater than the last-seen max. Animate new events with a pop-in CSS keyframe. Show a green pulsing dot in the header when live mode is on. Don’t re-render the entire SVG; only append new circles and arrows.”

Prompt 3 (crash resilience):

“The parser crashes when a JSONL file has a truncated last line, which happens when a node is killed mid-write. Wrap each `JSON.parse` in a `try/catch` and skip malformed lines silently. Also add a small toast notification showing how many lines were skipped on the last load.”